

```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.P03.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S03_Main";
    }
    /// <summary>
    /// S00_Boot      Mock → →      T001④
    ///      Boot      UI T001b  Boot      =
    /// </summary>
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null && !op.isDone) yield return null;
        }
    }
}

// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.P03.Boot
{
    /// <summary>
```

```

/// Boot      T001
/// ①Logger→② →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦      [BOOT] n/7
/// ⑦      Play  =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
/// X006      SY MOCK
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &= Run(1, "Logger", LoggerStep);
        ok &= Run(2, "Time", TimeStep);
        ok &= Run(3, "Config", ConfigStep);
        ok &= Run(4, "Save", SaveStep);
        ok &= Run(5, "ILocale", LocaleStep);
        ok &= Run(6, "Ads.Preload", AdsStep);
        ok &= Run(7, "SwitchMain", () =&gt; _switchScene != null && _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; //      JS sink platform T001c
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 && !time.TimeCheated();
    }
}

```

```

        static bool ConfigStep()
        {
            var config = ServiceLocator.Get<IConfigService>();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock= X009 manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get<ISaveService>();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get<ILocaleService>();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); // +
        }
        static bool AdsStep()
        {
            ServiceLocator.Get<IAdsService>().Preload("boot_warmup");
            return true;
        }
    }
}

// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P03.Boot
{
    /// <summary>
    /// Mock SY MOCK X006 Mock X002a AppID touristappid
    /// T001c WX/TT Channel Editor Boot
    /// </summary>
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet<ITimeService>(out _)) return; //
            ServiceLocator.Register<ITimeService>(new MockTimeService());
            ServiceLocator.Register<IConfigService>(new MockConfigService());
            ServiceLocator.Register<ISaveService>(new MockSaveService());
            ServiceLocator.Register<ILocaleService>(new MockLocaleService());
            ServiceLocator.Register<IAdsService>(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY MOCK)");
        }
    }
}

```

```

}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source => TimeSource.Mock;
    public long ClientNowMs() => DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// <summary>SY MOCK X006 TCB serverTime→HTTPS Date → T001b </summary>
    public long ServerNowMs() => ClientNowMs();
    public bool TimeCheated() => Math.Abs(ServerNowMs() - ClientNowMs()) > 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable<T>(string tableId) where T : class
        => throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable<T>(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        => new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) => "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) => PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) => PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// <summary>X006 T001b v1→v2 </summary>
    public SyncResult CloudSync() => new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// <summary>key key T001b cfg_i18n + NUnit </summary>
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        => new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action<AdLifecycle> OnAdEvent;
    public Task<RewardResult> ShowRewarded(string pointId)
        => Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
}

```

```

        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}

// MainUI.cs
using UnityEngine;
using UnityEngine.SceneManagement;
namespace Sy.P03.Newbie
{
    public static class NewbieSceneIds
    {
        public const string Main = "S03_Main";
        public const string Battle = "S03_Battle";
    }
    /// <summary>S03          →S03_Battle M5          —          </summary>
    public sealed class MainUI : MonoBehaviour
    {
        public void StartBattle()
        {
            SceneManager.LoadScene(NewbieSceneIds.Battle);
        }
        void Awake()
        {
            Application.targetFrameRate = 30;
            Sy.Core.MainArtInjector.Apply(transform, "P03_scene_01", "P03_unit_01");
        }
    }
}

// BattleLauncher.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P03.Newbie
{
    /// <summary>
    /// S03_Battle          BattleSim+TutorialFlow+BattleView  Tick
    /// Sy.Telemetry Editor =Debug sink T001e          +TCB
    /// HUD          6
    /// </summary>
    public sealed class BattleLauncher : MonoBehaviour
    {
        BattleSim _sim;
        TutorialFlow _tutorial;
        BattleView _view;
        Sy.Telemetry.TelemetryService _telemetry;
        int _lastWave = -1, _lastLives = -1, _lastCrystals = -1, _lastKills = -1;
        void Awake()
        {

```

```
Application.targetFrameRate = 30;
var canvas = FindObjectOfType<Canvas>();
_sim = new BattleSim();
_view = gameObject.AddComponent<BattleView>();
_view.Initialize(canvas, _sim);
_telemetry = new Sy.Telemetry.TelemetryService();
_telemetry.AddSink(new TelemetryDebugSink());
_tutorial = new TutorialFlow(
    _sim,
    hint => {
        Debug.Log("[HINT] " + hint);
        _view.SetHint(hint);
    },
    step => { _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
        new Dictionary<string, string> { { "step", step.ToString() } });
    },
    _view.SlotTapped += OnSlotTapped;
    _view.TianjiPicked += OnTianjiPicked;
    _view.QingLongRequested += OnQingLongRequested;
    _view.EnemyTapped += OnEnemyTapped;
    _sim.TowerPlaced += OnTowerPlaced;
    _sim.TianjiOffered += OnTianjiOffered;
    _sim.QingLongFired += OnQingLongFired;
    _sim.StateChanged += OnStateChanged;
}

void Update()
{
    _sim.Tick(Time.deltaTime);
    _view.SyncView(_sim);
    if (_sim.Wave != _lastWave) { _lastWave = _sim.Wave; _view.SetWave(_lastWave); }
    if (_sim.Lives != _lastLives) { _lastLives = _sim.Lives; _view.SetLives(_lastLives); }
    if (_sim.Crystals != _lastCrystals) { _lastCrystals = _sim.Crystals; _view.SetCrystals(_lastCrystals); }
    if (_sim.Kills != _lastKills) { _lastKills = _sim.Kills; _view.SetKills(_lastKills); }
}

void OnSlotTapped(int slot)
{
    _sim.PlaceTower(slot);
}

void OnTianjiPicked(int index)
{
    _sim.PickTianji(index);
}

void OnQingLongRequested()
{
    _sim.TryQingLongZhan();
}

void OnEnemyTapped(int enemyId)
{
    _sim.TapFocus(enemyId);
}
```

```

void OnTowerPlaced(BattleSim.TowerInst tower, int slot)
{
    _view.SetTower(slot, tower.IsGuanYu);
}
void OnTianjiOffered()
{
    _view.ShowTianjiPanel();
}
void OnQingLongFired()
{
    _view.PlaySlash();
}
void OnStateChanged(BattleSim.BattleState state)
{
    switch (state)
    {
        case BattleSim.BattleState.Prepare:
            _view.HideIntro();
            break;
        case BattleSim.BattleState.Won:
            _view.ShowSettle(_sim.Wave, _sim.Kills, _sim.Crystals);
            break;
        case BattleSim.BattleState.Lost:
            _view.ShowSettle(-1, _sim.Kills, _sim.Crystals);
            break;
    }
}
sealed class TelemetryDebugSink : Sy.Telemetry.TelemetryService.ISink
{
    public void Write(string evtId, Dictionary<string, string> fields)
    {
        var sb = new System.Text.StringBuilder(evtId);
        foreach (var kv in fields)
        {
            sb.Append(' ').Append(kv.Key).Append('=').Append(kv.Value);
        }
        Debug.Log("[TELEMETRY] " + sb);
    }
}
}

// BattleSim.cs
using System;
using System.Collections.Generic;
namespace Sy.P03.Newbie
{
    /// <summary>
    ///         C#      Play  Editor headless  NewbieFlowCheck
    ///         06  03  M0      →M1 L1          x2+  x1 →M2  +
    /// →M3          →M4  Boss+          →M5  +

```

```
///      A* S          =
/// &lt;/summary&gt;
public sealed class BattleSim
{
    public enum BattleState { Intro, Prep, Fighting, TianjiPause, Won, Lost }
    public sealed class TowerInst
    {
        public int Id;
        public int Slot;
        public bool IsGuanYu;
        public float Range;
        public float Atk;
        public float Interval;
        public float Cooldown;
    }
    public sealed class EnemyInst
    {
        public int Id;
        public bool Boss;
        public int Seg;
        public float SegT;
        public float X, Y;
        public float Hp, MaxHp;
        public float Speed;
        public bool Dead;
    }
    public BattleState State { get; private set; } = BattleState.Intro;
    readonly List<TowerInst> _towers = new();
    readonly List<EnemyInst> _enemies = new();
    readonly float[] _segLens;
    int _nextId = 1;
    float _stateTimer;
    int _placed;
    int _waveSpawned;
    float _spawnTimer;
    bool _spawning;
    float _waveGapTimer;
    int _lastCleared = -1;
    bool _tianjiDone;
    int _focusId = -1;
    float _focusTimer;
    float _enemySpeedMul = 1f;
    float _atkMul = 1f;
    int _waveBonus;
    public IReadOnlyList<TowerInst> Towers => _towers;
    public IReadOnlyList<EnemyInst> Enemies => _enemies;
    public int Wave { get; private set; }
    public int Lives { get; private set; } = NewbieConfig.StartLives;
    public int Crystals { get; private set; }
    public int Kills { get; private set; }
```

```
public int PlacedCount =&gt; _placed;
public float BattleTime { get; private set; }
public event Action<TowerInst, int> TowerPlaced;
public event Action<int> WaveStarted;
public event Action<int> WaveCleared;
public event Action<EnemyInst> EnemySpawned;
public event Action<EnemyInst> EnemyKilled;
public event Action<int> LifeLost;
public event Action TianjiOffered;
public event Action<int> TianjiPicked;
public event Action BossKilled;
public event Action QingLongFired;
public event Action<BattleState> StateChanged;
public BattleSim()
{
    _segLens = new float[NewbieConfig.PathX.Length - 1];
    for (int s = 0; s < _segLens.Length; s++)
    {
        float dx = NewbieConfig.PathX[s + 1] - NewbieConfig.PathX[s];
        float dy = NewbieConfig.PathY[s + 1] - NewbieConfig.PathY[s];
        _segLens[s] = (float)Math.Sqrt(dx * dx + dy * dy);
    }
}
/// <summary>          → →          </summary>
public bool PlaceTower(int slot)
{
    if (State != BattleState.Prep) return false;
    if (slot < 0 || slot >= NewbieConfig.SlotX.Length) return false;
    for (int i = 0; i < _towers.Count; i++)
        if (_towers[i].Slot == slot) return false;
    if (_placed >= NewbieConfig.FreeTowerPlacements) return false;
    bool isGuanYu = _placed < NewbieConfig.FreeTowerPlacements - 1; //
    var def = isGuanYu ? NewbieConfig.GuanYu : NewbieConfig.HuangZhong;
    var tower = new TowerInst
    {
        Id = _nextId++,
        Slot = slot,
        IsGuanYu = isGuanYu,
        Range = def.Range,
        Atk = def.Atk,
        Interval = def.Interval,
    };
    _towers.Add(tower);
    _placed++;
    TowerPlaced?.Invoke(tower, slot);
    return true;
}
/// <summary>          FocusSeconds </summary>
public void TapFocus(int enemyId)
{
```

```
        for (int i = 0; i < _enemies.Count; i++)
        {
            if (_enemies[i].Id == enemyId)
            {
                _focusId = enemyId;
                _focusTimer = NewbieConfig.FocusSeconds;
                return;
            }
        }
    }

    public bool IsFocused(EnemyInst e) => e.Id == _focusId && _focusTimer > 0f;
    /// <summary> ≤40% → </summary>
    public bool QingLongZhanReady
    {
        get
        {
            for (int i = 0; i < _enemies.Count; i++)
            {
                var e = _enemies[i];
                if (e.Boss && e.Hp <= e.MaxHp * NewbieConfig.QingLongHpRatio) return true;
            }
            return false;
        }
    }
    /// <summary> M4 . </summary>
    public bool TryQingLongZhan()
    {
        if (State != BattleState.Fighting || !QingLongZhanReady) return false;
        for (int i = 0; i < _enemies.Count; i++)
        {
            var e = _enemies[i];
            if (!e.Boss) continue;
            QingLongFired?.Invoke();
            DamageEnemy(e, e.Hp + 1f);
            return true;
        }
        return false;
    }

    public void PickTianji(int index)
    {
        if (State != BattleState.TianjiPause) return;
        switch (index)
        {
            case 0: _enemySpeedMul = 0.8f; break;
            case 1: _atkMul = 1.25f; break;
            default: _waveBonus = 10; break;
        }
        _waveGapTimer = 0f;
        TianjiPicked?.Invoke(index);
        SetState(BattleState.Fighting);
    }

    public void Tick(float dt)
```

```
{
    switch (State)
    {
        case BattleState.Intro:
            _stateTimer += dt;
            if (_stateTimer >= NewbieConfig.IntroSeconds) SetState(BattleState.Prep);
            return;
        case BattleState.Prep:
            _stateTimer += dt;
            if (_placed >= 1 && (_placed >= NewbieConfig.FreeTowerPlacements || _stateTimer >=
            {
                SetState(BattleState.Fighting);
                BeginWave(1);
            }
            return;
        case BattleState.Fighting:
            BattleTime += dt;
            if (_focusTimer > 0f) _focusTimer -= dt;
            TickSpawns(dt);
            TickEnemies(dt);
            TickTowers(dt);
            TickWaves(dt);
            if (Lives <= 0) SetState(BattleState.Lost);
            return;
    }
}

void BeginWave(int w)
{
    Wave = w;
    var (count, __, _) = NewbieConfig.GetWave(w);
    _waveSpawned = 0;
    _spawnTimer = 0f;
    _waveGapTimer = 0f;
    _spawning = true;
    WaveStarted?.Invoke(w);
    if (w == NewbieConfig.NewbieWaves)
    {
        var boss = new EnemyInst
        {
            Id = _nextId++,
            Boss = true,
            Seg = 0,
            SegT = 0f,
            X = NewbieConfig.PathX[0],
            Y = NewbieConfig.PathY[0],
            Hp = NewbieConfig.BossHp,
            MaxHp = NewbieConfig.BossHp,
            Speed = NewbieConfig.BossSpeed,
        };
        _enemies.Add(boss);
    }
}
```

```
        EnemySpawned?.Invoke(boss);
    }
}

void TickSpawns(float dt)
{
    if (!_spawning) return;
    var (count, hp, speed) = NewbieConfig.GetWave(Wave);
    _spawnTimer += dt;
    while (_spawnTimer >= NewbieConfig.SpawnInterval && _waveSpawned < count)
    {
        _spawnTimer -= NewbieConfig.SpawnInterval;
        _waveSpawned++;
        var e = new EnemyInst
        {
            Id = _nextId++,
            Seg = 0,
            SegT = 0f,
            X = NewbieConfig.PathX[0],
            Y = NewbieConfig.PathY[0],
            Hp = hp,
            MaxHp = hp,
            Speed = speed,
        };
        _enemies.Add(e);
        EnemySpawned?.Invoke(e);
    }
    if (_waveSpawned >= count) _spawning = false;
}

void TickEnemies(float dt)
{
    for (int i = _enemies.Count - 1; i >= 0; i--)
    {
        var e = _enemies[i];
        float move = e.Speed * _enemySpeedMul * dt;
        while (move > 0f)
        {
            float segLen = _segLens[e.Seg];
            float remain = (1f - e.SegT) * segLen;
            if (move < remain)
            {
                e.SegT += move / segLen;
                move = 0f;
            }
            else
            {
                move -= remain;
                e.Seg++;
                e.SegT = 0f;
                if (e.Seg >= _segLens.Length)
                {

```

```
        if (e.Boss)
        {
            //      =
            e.Seg = 0;
            e.SegT = 0f;
            Lives = Math.Max(0, Lives - 1);
            LifeLost?.Invoke(Lives);
        }
        else
        {
            _enemies.RemoveAt(i);
            Lives = Math.Max(0, Lives - 1);
            LifeLost?.Invoke(Lives);
        }
        break;
    }
}

}

if (e.Seg < _segLens.Length)
{
    e.X = Lerp(NewbieConfig.PathX[e.Seg], NewbieConfig.PathX[e.Seg + 1], e.SegT);
    e.Y = Lerp(NewbieConfig.PathY[e.Seg], NewbieConfig.PathY[e.Seg + 1], e.SegT);
}
}

}

static float Lerp(float a, float b, float t) => a + (b - a) * t;
void TickTowers(float dt)
{
    for (int i = 0; i < _towers.Count; i++)
    {
        var t = _towers[i];
        t.Cooldown -= dt;
        if (t.Cooldown > 0f) continue;
        bool any = false;
        if (t.IsGuanYu)
        {
            //
            float r2 = t.Range * t.Range;
            for (int j = _enemies.Count - 1; j >= 0; j--)
            {
                var e = _enemies[j];
                float dx = e.X - NewbieConfig.SlotX[t.Slot], dy = e.Y - NewbieConfig.SlotY[t.Slot];
                if (dx * dx + dy * dy > r2) continue;
                any = true;
                DamageEnemy(e, t.Atk * _atkMul);
            }
        }
        else
        {
            //

```

```
        float r2 = t.Range * t.Range;
        EnemyInst target = null;
        float best = float.MaxValue;
        for (int j = 0; j < _enemies.Count; j++)
        {
            var e = _enemies[j];
            float dx = e.X - NewbieConfig.SlotX[t.Slot], dy = e.Y - NewbieConfig.SlotY[t.Slot];
            float d2 = dx * dx + dy * dy;
            if (d2 > r2) continue;
            if (IsFocused(e)) { target = e; best = -1f; break; }
            if (d2 < best) { best = d2; target = e; }
        }
        if (target != null)
        {
            any = true;
            DamageEnemy(target, t.Atk * _atkMul);
        }
    }
    t.Cooldown = any ? t.Interval : 0.05f;
}

void DamageEnemy(EnemyInst e, float dmg)
{
    e.Hp -= dmg;
    if (e.Hp > 0f) return;
    e.Dead = true;
    _enemies.Remove(e);
    Kills++;
    Crystals += NewbieConfig.KillCrystal;
    if (e.Boss)
    {
        //
        for (int i = _enemies.Count - 1; i >= 0; i--)
        {
            var rest = _enemies[i];
            rest.Dead = true;
            _enemies.RemoveAt(i);
            Kills++;
            Crystals += NewbieConfig.KillCrystal;
            EnemyKilled?.Invoke(rest);
        }
        BossKilled?.Invoke();
        EnemyKilled?.Invoke(e);
        SetState(BattleState.Won);
        return;
    }
    EnemyKilled?.Invoke(e);
}

void TickWaves(float dt)
{

```

```

        if (_spawning || _enemies.Count > 0) return;
        _waveGapTimer += dt;
        if (_waveGapTimer <= NewbieConfig.WaveGapSeconds) return;
        int cleared = Wave;
        if (cleared >= NewbieConfig.NewbieWaves) return;    //    Boss
        if (_lastCleared != cleared)
        {
            _lastCleared = cleared;
            Crystals += NewbieConfig.WaveClearCrystal + _waveBonus;
            WaveCleared?.Invoke(cleared);
        }
        if (cleared == 2 && !_tianjiDone)
        {
            _tianjiDone = true;
            SetState(BattleState.TianjiPause);
            TianjiOffered?.Invoke();
            return;
        }
        BeginWave(cleared + 1);
    }
    void SetState(BattleState s)
    {
        if (State == s) return;
        State = s;
        _stateTimer = 0f;
        StateChanged?.Invoke(s);
    }
}

// BattleView.cs
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P03.Newbie
{
    /// <summary>
    ///     UGUI—  x  x  x
    ///  UI Initialize      YAML      =      /      /      /
    ///
    /// </summary>
    public sealed class BattleView : MonoBehaviour
    {
        Canvas _canvas;
        Sprite _square;
        readonly Image[] _slotImages = new Image[NewbieConfig.SlotX.Length];
        readonly Text[] _slotLabels = new Text[NewbieConfig.SlotX.Length];
        Text _waveText, _livesText, _crystalText, _killsText, _hintText;
        Button _qingLongButton;
    }
}

```

```

Image _qingLongImage;
RectTransform _introOverlay, _tianjiPanel, _slashOverlay, _settlePanel;
Text _settleStats;
readonly Dictionary<int, EnemyVisual> _enemies = new();
readonly List<int> _scratch = new();
public event Action<int> SlotTapped;
public event Action<int> EnemyTapped;
public event Action<int> TianjiPicked;
public event Action QingLongRequested;
static readonly Color RevRed = new Color(0.757f, 0.153f, 0.176f); // #C1272D
static readonly Color PaperYellow = new Color(0.929f, 0.871f, 0.706f); //
static readonly Color InkBlack = new Color(0.043f, 0.043f, 0.047f); //
static readonly Color DoomGreen = new Color(0.298f, 0.686f, 0.314f); // #4CAF50
static readonly Color PathCol = new Color(0.16f, 0.08f, 0.08f);
public void Initialize(Canvas canvas, BattleSim sim)
{
    _canvas = canvas;
    _square = MakeSquareSprite();
    BuildBackdrop();
    BuildHud();
    BuildPath();
    BuildSlots();
    BuildIntroOverlay();
    BuildTianjiPanel();
    BuildSlashOverlay();
    BuildSettlePanel();
    SetHint(" .....");
}
// —— ——
/// <summary>X027 generated art: fullscreen backdrop behind all UI; skipped
/// entirely when the art key is missing so headless checks stay green.</summary>
void BuildBackdrop()
{
    var art = Sy.Core.ArtRuntime.Get("P03_scene_01");
    if (art == null) return;
    var go = new GameObject("ArtBackdrop", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    rt.SetAsFirstSibling();
    var image = go.GetComponent<Image>();
    image.sprite = art;
    image.color = Color.white;
    image.preserveAspect = true;
    image.raycastTarget = false;
}
void BuildHud()
{

```

```

var root = _canvas.transform;
_waveText = MakeText(root, "L1    0/4", 46, new Vector2(0, 880), TextAnchor.UpperLeft, 380, 70);
_waveText.color = PaperYellow;
_livesText = MakeText(root, "    5", 46, new Vector2(0, 880), TextAnchor.UpperRight, 340, 70);
_crystalText = MakeText(root, "    0", 44, new Vector2(0, 820), TextAnchor.UpperLeft, 380, 60);
_crystalText.color = PaperYellow;
_killsText = MakeText(root, "    0", 44, new Vector2(0, 820), TextAnchor.UpperRight, 380, 60);
_hintText = MakeText(root, "", 40, new Vector2(0, -780), TextAnchor.MiddleCenter, 1020, 150);
_hintText.color = new Color(0.95f, 0.87f, 0.6f);
var btnGo = new GameObject("QingLongButton", typeof(Image), typeof(Button));
btnGo.transform.SetParent(root, false);
var btnRt = (RectTransform)btnGo.transform;
btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0f);
btnRt.anchoredPosition = new Vector2(0, 120);
btnRt.sizeDelta = new Vector2(460, 140);
_qingLongImage = btnGo.GetComponent<Image>();
_qingLongImage.sprite = _square;
_qingLongImage.color = new Color(0.35f, 0.35f, 0.35f);
_qingLongButton = btnGo.GetComponent<Button>();
_qingLongButton.interactable = false;
MakeText(btnGo.transform, "    ", 54, Vector2.zero, TextAnchor.MiddleCenter, 460, 140).color = InkBlack;
_qingLongButton.onClick.AddListener(() => QingLongRequested?.Invoke());
}
void BuildPath()
{
    for (int s = 0; s < NewbieConfig.PathX.Length - 1; s++)
    {
        float x0 = NewbieConfig.PathX[s], y0 = NewbieConfig.PathY[s];
        float x1 = NewbieConfig.PathX[s + 1], y1 = NewbieConfig.PathY[s + 1];
        bool horiz = Math.Abs(y1 - y0) < 1f;
        var go = new GameObject("PathSeg_" + s, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2((x0 + x1) * 0.5f, (y0 + y1) * 0.5f);
        rt.sizeDelta = horiz
            ? new Vector2(Math.Abs(x1 - x0) + 90f, 90f)
            : new Vector2(90f, Math.Abs(y1 - y0) + 90f);
        var img = go.GetComponent<Image>();
        img.sprite = _square;
        img.color = PathCol;
    }
    MakeText(_canvas.transform, "    ", 34, new Vector2(-460f, 820f), TextAnchor.MiddleCenter, 260, 50).color =
    MakeText(_canvas.transform, "    ", 34, new Vector2(-500f, -660f), TextAnchor.MiddleCenter, 260, 50).color = P
}
void BuildSlots()
{
    for (int i = 0; i < NewbieConfig.SlotX.Length; i++)
    {
        int index = i;

```

```

        var go = new GameObject("Slot_" + i, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2(NewbieConfig.SlotX[i], NewbieConfig.SlotY[i]);
        rt.sizeDelta = new Vector2(140, 140);
        var img = go.GetComponent<Image>();
        var slotArt = Sy.Core.ArtRuntime.Get("P03_unit_01");
        if (slotArt != null)
        {
            img.sprite = slotArt;
            img.color = Color.white;
            img.preserveAspect = true;
        }
        else
        {
            img.sprite = _square;
            img.color = new Color(0.13f, 0.13f, 0.13f, 0.9f);
        }
        _slotImages[i] = img;
        _slotLabels[i] = MakeText(go.transform, " ", 34, Vector2.zero, TextAnchor.MiddleCenter, 140, 140);
        _slotLabels[i].color = new Color(0.55f, 0.52f, 0.45f);
        go.GetComponent<Button>().onClick.AddListener(() => SlotTapped?.Invoke(index));
    }
}

void BuildIntroOverlay()
{
    _introOverlay = MakeOverlayPanel("IntroOverlay", new Color(0.06f, 0.02f, 0.02f, 0.95f));
    MakeText(_introOverlay, " ", 100, new Vector2(0, 100), TextAnchor.MiddleCenter, 1000, 130).color = Rev
    MakeText(_introOverlay, " · L1 ", 46, new Vector2(0, -60), TextAnchor.MiddleCenter, 1000, 80).color =
}

void BuildTianjiPanel()
{
    _tianjiPanel = MakeOverlayPanel("TianjiPanel", new Color(0.03f, 0.03f, 0.03f, 0.92f));
    MakeText(_tianjiPanel, " · ", 76, new Vector2(0, 560), TextAnchor.MiddleCenter, 1000, 100).color = Pape
    for (int i = 0; i < NewbieConfig.TianjiCards.Length; i++)
    {
        int index = i;
        var card = NewbieConfig.TianjiCards[i];
        var go = new GameObject("Tianji_" + i, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(_tianjiPanel, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2(-330 + i * 330, 0);
        rt.sizeDelta = new Vector2(300, 420);
        var img = go.GetComponent<Image>();
        img.sprite = _square;
        img.color = i == 0 ? new Color(0.55f, 0.12f, 0.12f) : new Color(0.14f, 0.16f, 0.14f); //
        var label = MakeText(go.transform, " " + card.Name + " ", 56, new Vector2(0, 120), TextAnchor.MiddleCenter
        label.color = PaperYellow;
    }
}

```

```

        MakeText(go.transform, card.Desc, 38, new Vector2(0, -40), TextAnchor.MiddleCenter, 280, 200).color = ne
        go.GetComponent<Button>().onClick.AddListener(() =>{
            {
                _tianjiPanel.gameObject.SetActive(false);
                TianjiPicked?.Invoke(index);
            });
        }
        MakeText(_tianjiPanel, "      —      ", 40, new Vector2(0, -400), TextAnchor.MiddleCenter, 1000, 80).color
        _tianjiPanel.gameObject.SetActive(false);
    }
    void BuildSlashOverlay()
    {
        _slashOverlay = MakeOverlayPanel("SlashOverlay", new Color(1f, 0.98f, 0.92f, 0.75f));
        var lineGo = new GameObject("SlashLine", typeof(RectTransform), typeof(Image));
        lineGo.transform.SetParent(_slashOverlay, false);
        var lineRt = (RectTransform)lineGo.transform;
        lineRt.anchorMin = lineRt.anchorMax = new Vector2(0.5f, 0.5f);
        lineRt.anchoredPosition = Vector2.zero;
        lineRt.sizeDelta = new Vector2(2200, 16);
        lineRt.localRotation = Quaternion.Euler(0f, 0f, 24f);
        var lineImg = lineGo.GetComponent<Image>();
        lineImg.sprite = _square;
        lineImg.color = RevRed;
        MakeText(_slashOverlay, "      ", 110, new Vector2(0, -180), TextAnchor.MiddleCenter, 1000, 140).color = Ink
        _slashOverlay.gameObject.SetActive(false);
    }
    void BuildSettlePanel()
    {
        _settlePanel = MakeOverlayPanel("SettlePanel", new Color(0.03f, 0.03f, 0.03f, 0.96f));
        MakeText(_settlePanel, "L1      ", 90, new Vector2(0, 560), TextAnchor.MiddleCenter, 1040, 110).color = Paper
        _settleStats = MakeText(_settlePanel, "", 46, new Vector2(0, 380), TextAnchor.MiddleCenter, 1000, 260);
        MakeActionButton(_settlePanel, "      ", new Vector2(-260, -260), () => UnityEngine.SceneManagement.SceneMa
        MakeActionButton(_settlePanel, "      ", new Vector2(260, -260), () => UnityEngine.SceneManagement.SceneMa
        _settlePanel.gameObject.SetActive(false);
    }
    // ————— reconcile —————
    public void SyncView(BattleSim sim)
    {
        ReconcileEnemies(sim);
        foreach (var kv in _enemies)
        {
            var v = kv.Value;
            v.Rt.anchoredPosition = new Vector2(v.X, v.Y);
            if (v.Fill != null) v.Fill.fillAmount = v.MaxHp > 0f ? v.Hp / v.MaxHp : 0f;
            bool focused = v.Focused;
            if (focused != v.LastFocused)
            {
                v.LastFocused = focused;
                v.FocusLabel.gameObject.SetActive(focused);
            }
        }
    }

```

```
    }
    bool ready = sim.QingLongZhanReady;
    if (_qingLongButton.interactable != ready)
    {
        _qingLongButton.interactable = ready;
        _qingLongImage.color = ready ? RevRed : new Color(0.35f, 0.35f, 0.35f);
    }
}

void ReconcileEnemies(BattleSim sim)
{
    _scratch.Clear();
    foreach (var kv in _enemies)
        if (!ContainsEnemy(sim, kv.Key)) _scratch.Add(kv.Key);
    for (int i = 0; i < _scratch.Count; i++)
    {
        Destroy(_enemies[_scratch[i]].Go);
        _enemies.Remove(_scratch[i]);
    }
    for (int i = 0; i < sim.Enemies.Count; i++)
    {
        var e = sim.Enemies[i];
        if (!_enemies.TryGetValue(e.Id, out var v))
        {
            v = CreateEnemyVisual(e);
            _enemies[e.Id] = v;
        }
        v.X = e.X;
        v.Y = e.Y;
        v.Hp = e.Hp;
        v.MaxHp = e.MaxHp;
        v.Focused = sim.IsFocused(e);
    }
}

static bool ContainsEnemy(BattleSim sim, int id)
{
    for (int i = 0; i < sim.Enemies.Count; i++)
        if (sim.Enemies[i].Id == id) return true;
    return false;
}

EnemyVisual CreateEnemyVisual(BattleSim.EnemyInst e)
{
    int id = e.Id;
    var go = new GameObject("Enemy_" + e.Id, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.sizeDelta = e.Boss ? new Vector2(160, 160) : new Vector2(70, 70);
    var image = go.GetComponent<Image>();
    image.sprite = _square;
    image.color = e.Boss ? PaperYellow : new Color(0.35f, 0.42f, 0.33f);
}
```

```

    if (e.Boss)
        MakeText(go.transform, " ", 64, Vector2.zero, TextAnchor.MiddleCenter, 160, 160).color = InkBlack;
    Image fill = null;
    var hpGo = new GameObject("Hp", typeof(RectTransform), typeof(Image));
    hpGo.transform.SetParent(go.transform, false);
    var hpRt = (RectTransform)hpGo.transform;
    hpRt.anchorMin = new Vector2(0f, 1f);
    hpRt.anchorMax = new Vector2(1f, 1f);
    hpRt.anchoredPosition = new Vector2(0, -6);
    hpRt.sizeDelta = new Vector2(-8, 10);
    fill = hpGo.GetComponent<Image>();
    fill.sprite = _square;
    fill.type = Image.Type.Filled;
    fill.fillMethod = Image.FillMethod.Horizontal;
    fill.color = new Color(0.2f, 0.9f, 0.3f);
    var focusLabel = MakeText(go.transform, " ", 34, new Vector2(0, 120), TextAnchor.MiddleCenter, 220, 50);
    focusLabel.color = PaperYellow;
    focusLabel.gameObject.SetActive(false);
    go.GetComponent<Button>().onClick.AddListener(() => EnemyTapped?.Invoke(id));
    return new EnemyVisual
    {
        Go = go,
        Rt = rt,
        Fill = fill,
        FocusLabel = focusLabel.gameObject,
        X = e.X,
        Y = e.Y,
        Hp = e.Hp,
        MaxHp = e.MaxHp,
    };
}

// —— HUD ——

public void SetWave(int w) => _waveText.text = "L1 " + w + "/" + NewbieConfig.NewbieWaves;
public void SetLives(int l) => _livesText.text = " " + l;
public void SetCrystals(int c) => _crystalText.text = " " + c;
public void SetKills(int k) => _killsText.text = " " + k;
public void SetHint(string hint) => _hintText.text = hint;
public void HideIntro() => _introOverlay.gameObject.SetActive(false);
public void ShowTianjiPanel() => _tianjiPanel.gameObject.SetActive(true);
public void SetTower(int slot, bool isGuanYu)
{
    _slotImages[slot].color = isGuanYu ? RevRed : DoomGreen;
    _slotLabels[slot].text = isGuanYu ? " " : " ";
    _slotLabels[slot].color = InkBlack;
    _slotLabels[slot].fontSize = 64;
}

public void PlaySlash()
{
    StartCoroutine(SlashRoutine());
}

```

```

IEnumerator SlashRoutine()
{
    _slashOverlay.gameObject.SetActive(true);
    yield return new WaitForSeconds(0.5f); // 06 M4
    _slashOverlay.gameObject.SetActive(false);
}
public void ShowSettle(int wave, int kills, int crystals)
{
    _settlePanel.gameObject.SetActive(true);
    _settleStats.text = wave &gt;= 0
        ? $" {wave}/{NewbieConfig.NewbieWaves} · {kills} · {crystals}\n&lt;size=40&gt; L1-DEMO-0001
        : $" ..... {kills} · {crystals}\n&lt;size=40&gt; — &lt;/size&gt;";
}
// —— ——
RectTransform MakeOverlayPanel(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = color;
    return rt;
}
void MakeActionButton(RectTransform parent, string text, Vector2 pos, Action onClick)
{
    var go = new GameObject(text, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(420, 120);
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = DoomGreen;
    MakeText(go.transform, text, 46, Vector2.zero, TextAnchor.MiddleCenter, 420, 120);
    go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; onClick());
}
static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(w, h);
    var t = go.GetComponent&lt;Text&gt;();

```

```
t.font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
t.text = text;
t.fontSize = size;
t.alignment = anchor;
t.color = new Color(0.91f, 0.89f, 0.85f);
t.supportRichText = true;
return t;
}

static Sprite MakeSquareSprite()
{
    var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
    var px = tex.GetPixels32();
    for (int i = 0; i < px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
    tex.SetPixels32(px);
    tex.Apply(false, false);
    return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
}

// -----
sealed class EnemyVisual
{
    public GameObject Go;
    public RectTransform Rt;
    public Image Fill;
    public GameObject FocusLabel;
    public bool LastFocused;
    public bool Focused;
    public float X, Y;
    public float Hp, MaxHp;
}

}

// NewbieConfig.cs
namespace Sy.P03.Newbie
{
    public readonly struct TowerDef
    {
        public readonly string Name;
        public readonly float Range;
        public readonly float Atk;
        public readonly float Interval;
        public readonly bool AoE; // =
        public TowerDef(string name, float range, float atk, float interval, bool aoe)
        {
            Name = name;
            Range = range;
            Atk = atk;
            Interval = interval;
            AoE = aoe;
        }
    }
}
```

```

/// <summary>
/// P03 . V0.1— X009+IConfigService Design/configs/P03 JSON
/// 06 03 M0-M5 /L1 / / /
/// #C1272D x x x #4CAF50 A* S — AI
/// </summary>
public static class NewbieConfig
{
    public const string GameTitle = " ";
    public const string GameSlogan = " . ";
    public const int NewbieWaves = 4; // L1 4 W4= Boss
    public const int StartLives = 5; // -1 /
    public const float IntroSeconds = 3f; // M0
    public const float PrepStartSeconds = 15f; // 1
    public const int FreeTowerPlacements = 3; // x2+ x1
    public const float SpawnInterval = 0.8f;
    public const float WaveGapSeconds = 2f;
    public const float MaxTutorialSeconds = 210f; // ≤3.5min 06 ②
    public const int KillCrystal = 2; // +2/ +8
    public const int WaveClearCrystal = 8;
    /// <summary> / </summary>
    public static readonly TowerDef GuanYu = new(" ", range: 280f, atk: 12f, interval: 0.8f, aoe: true);
    public static readonly TowerDef HuangZhong = new(" ", range: 520f, atk: 26f, interval: 1.1f, aoe: false);
    /// <summary>S 8 7 → </summary>
    public static readonly float[] PathX = { -460f, 340f, 340f, -340f, -340f, 340f, 340f, -500f };
    public static readonly float[] PathY = { 740f, 740f, 300f, 300f, -140f, -140f, -580f, -580f };
    /// <summary>8 3 = / / </summary>
    public static readonly float[] SlotX = { 0f, 0f, 0f, -320f, 320f, -320f, 320f, -420f };
    public static readonly float[] SlotY = { 620f, 420f, -280f, 620f, 420f, -280f, 620f, -280f };
    public const float BossHp = 900f;
    public const float BossSpeed = 55f;
    public const float QingLongHpRatio = 0.4f; // ≤40%→
    public const float FocusSeconds = 4f; //
    /// <summary> L1 W4=4 + </summary>
    public static (int Count, float Hp, float Speed) GetWave(int w) => w switch
    {
        1 => (5, 26f, 75f),
        2 => (7, 32f, 78f),
        3 => (10, 38f, 82f),
        _ => (4, 40f, 86f),
    };
    /// <summary> 06 M3 = </summary>
    public static readonly (string Name, string Desc)[] TianjiCards =
    {
        (" ", " -20% "),
        (" ", " +25% "),
        (" ", " +10"),
    };
}
}
// TutorialFlow.cs

```

```

using System;
namespace Sy.P03.Newbie
{
    /// <summary>
    ///     evt_newbie_step 1-5 ① ② ③ ④ Boss⑤ +
    ///     BattleSim / / / / —
    /// Play headless NewbieFlowCheck
    /// </summary>
    public sealed class TutorialFlow
    {
        public int Step { get; private set; }
        readonly Action<string> _hint;
        readonly Action<int> _onStep;
        bool _firstTowerSeen;
        public TutorialFlow(BattleSim sim, Action<string> hint, Action<int> onStep)
        {
            _hint = hint ?? delegate { };
            _onStep = onStep ?? delegate { };
            sim.TowerPlaced += OnTowerPlaced;
            sim.WaveCleared += OnWaveCleared;
            sim.TianjiOffered += OnTianjiOffered;
            sim.TianjiPicked += OnTianjiPicked;
            sim.BossKilled += OnBossKilled;
            sim.StateChanged += OnStateChanged;
            _hint(" —L1 x2 + x1 ");
        }
        void Advance(int step)
        {
            if (Step >= step) return;
            Step = step;
            _onStep(step);
        }
        void OnTowerPlaced(BattleSim.TowerInst tower, int slot)
        {
            if (!_firstTowerSeen)
            {
                _firstTowerSeen = true;
                Advance(1);
                _hint(" — ");
                return;
            }
            _hint(tower.IsGuanYu ? " — " : " — ");
        }
        void OnWaveCleared(int wave)
        {
            if (wave == 1)
            {
                Advance(2);
                _hint(" — ");
            }
        }
    }
}

```

```
    }
    void OnTianjiOffered()
    {
        _hint("      —      -20%");
    }
    void OnTianjiPicked(int index)
    {
        Advance(3);
        _hint(" " + NewbieConfig.TianjiCards[index].Name + "      " + NewbieConfig.TianjiCards[index].Desc + "—      ")
    }
    void OnBossKilled()
    {
        Advance(4);
        _hint("      —L1      ");
    }
    void OnStateChanged(BattleSim.BattleState state)
    {
        if (state == BattleSim.BattleState.Won)
        {
            Advance(5);
            _hint("L1      —      ");
        }
    }
}
}
```